Computer Lab: SQL Server-7

Outer joins

The join operation combines data from two tables by forming pairs of related rows where the matching columns in each table have the same value.

- If one row of a table is unmatched, the row is omitted from the result table. This has been the case for the joins we examined above.
- The ISO standard provides another set of join operators called **outer joins**.
- The Outer join retains rows that do not satisfy the join condition.
- To understand the Outer join operators, consider the following two simplified Branch and Project tables, which we refer to as Branch1 and Project1, respectively:

Branch1

branchNo	city
B01	Banha
B02	Cairo
B03	Mansoura

Project1

projNo	city
PT11	Zakazik
PT21	Mansoura
PT32	Banha

Ex 7.27

The (**Inner**) join of these two tables:

```
SELECT b.*, p.*
FROM Branch1 b, Project1 p
WHERE b.bCity = p.pCity;
```

produces the result table shown in Table 7.27.

Table 7.27 Result table for inner join of Branch1 and PropertyForRent1 tables.

branchNo	bcity	projNo	pcity
B01	Banha	PT32	Banha
B03	Mansoura	PT21	Mansoura

The result table has two rows where the cities are the same.

- Note that there is no row corresponding to the branch office in Cairo and there is no row corresponding to the project in Zakazik.
- If we want to include the unmatched rows in the result table, we can use an Outer join.
    - There are three types of Outer join: **Left**, **Right**, and **Full** Outer joins. We illustrate their functionality in the following examples.

Ex 7.28 Left Outer join

List all branch offices and any projects that are in the same city.

The Left Outer join of these two tables:

```
SELECT b.*, p.*
FROM Branch1 b LEFT JOIN Project1 p ON b.bCity = p.pCity;
```

produces the result table shown in Table 7.28.
In this example the Left Outer join includes not only those rows that have the same city, but also those rows of the first (left) table that are unmatched with rows from the second (right) table.
The columns from the second table are filled with NULLs.

Table 7.28 Result table for Example 7.28.

branchNo	bcity	projNo	pcity
B01	Banha	PT32	Banha
B02	Cairo	NULL	NULL
B03	Mansoura	PT21	Mansoura

Example 7.29 Right Outer join

List all projects and any branch offices that are in the same city.

The Right Outer join of these two tables:

```
SELECT b.*, p.*
FROM Branch1 b RIGHT JOIN Project1 p ON b.bCity = p.pCity;
```

produces the result table shown in Table 7.29.
In this example the Right Outer join includes not only those rows that have the same city, but also those rows of the second (right) table that are unmatched with rows from the first (left) table.
The columns from the first table are filled with NULLs.

Table 7.29 Result table for Example 7.29.

branchNo	bcity	projNo	pcity
NULL	NULL	PT11	Zakazik
B03	Mansoura	PT21	Mansoura
B01	Banha	PT32	Banha

Ex 7.30 Full Outer join

List the branch offices and projects that are in the same city along with any unmatched branches or projects.

The Full Outer join of these two tables:

```
SELECT b.*, p.*
FROM Branch1 b FULL JOIN Project1 p ON b.bCity = p.pCity;
```

produces the result table shown in Table 7.30.
In this case, the Full Outer join includes not only those rows that have the same city, but also those rows that are unmatched in both tables.
The unmatched columns are filled with NULLs.

Table 7.30 Result table for Example 7.30.

branchNo	bcity	projNo	pcity
NULL	NULL	PT11	Zakazik
B01	Banha	PT32	Banha
B02	Cairo	NULL	NULL
B03	Mansoura	PT21	Mansoura

ملخص

نوع الجوين	النتيجة في جدولك	الفائدة العملية
Inner Join سكشن 6	الموظفون الذين لديهم مشاريع فقط.	معرفة الانشغال الفعلي الحالي.
Left Outer Join	كل الموظفين (حتى الذين لا يعملون).	معرفة الموظفين "المتاحين" أو غير المسندين لعمل.
Right Outer Join	كل المشاريع (حتى التي ليس لها موظف).	معرفة المشاريع "المعطلة" أو التي تحتاج تعيين موظف.
Full Outer Join	كل الموظفين وكل المشاريع.	الحصول على صورة كاملة وشاملة لكل موارد الشركة (بشرية ومشاريع).

7.7 Combining Result Tables (UNION, INTERSECT, EXCEPT)

In SQL, we can use the normal set operations of *Union*, *Intersection*, and *Difference* to combine the results of two or more queries into a single result table

- The **Union** of two tables, A and B, is a table containing all rows that are in either the first table A or the second table B or both.
- The **Intersection** of two tables, A and B, is a table containing all rows that are common to both tables A and B.
- The **Difference** of two tables, A and B, is a table containing all rows that are in table A but are not in table B.

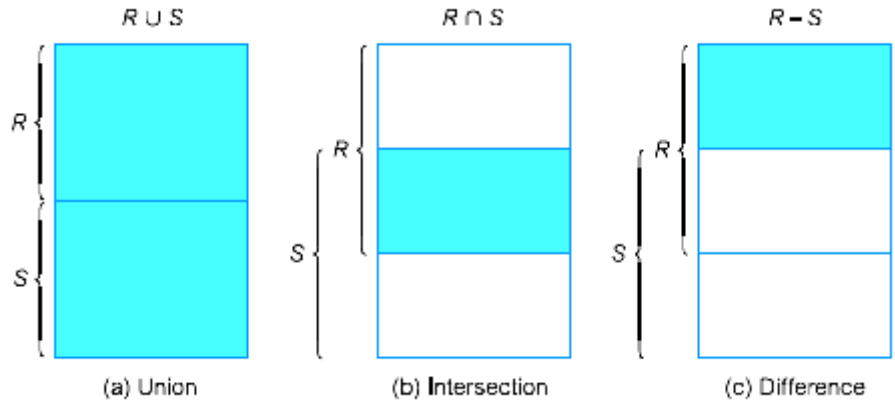

Figure 7.1 Union, intersection, and difference set operations.

The set operations are illustrated in Figure 7.1.

There are restrictions on the tables that can be combined using the set operations, the most important one being that the two tables must be **union-compatible**; that is, they have the same structure.

- This implies that the two tables must contain the same number of columns, and that their corresponding columns have the same data types and lengths.

- The three set operators in the ISO standard are called UNION, INTERSECT, and EXCEPT. The format of the set operator clause in each case is:

operator [**ALL**] [**CORRESPONDING** [**BY** {column1 [, . . .]}]]

If CORRESPONDING BY is specified, then the set operation is performed on the named column(s); if CORRESPONDING is specified but not the BY clause, the set operation is performed on the columns that are common to both tables. If ALL is specified, the result can include duplicate rows. Some dialects of SQL do not support INTERSECT and EXCEPT; others use MINUS in place of EXCEPT.

Ex 7.32 Use of UNION

Construct a list of all cities where there is either a branch office or a project.

(**SELECT** city
FROM Branch
WHERE city **IS NOT NULL**)
UNION
(**SELECT** city
FROM Project
WHERE city **IS NOT NULL**);

Or

(SELECT *

```
FROM Branch
WHERE city IS NOT NULL)
UNION CORRESPONDING BY city
(SELECT *
FROM Project
WHERE city IS NOT NULL);
```

This query is executed by producing a result table from the first query and a result table from the second query,

- and then merging both tables into a single result table consisting of all the rows from both result tables with the duplicate rows removed.
- The result table is shown in Table 7.32.

Table 7.32 Result table for Example 7.32.

city
Banha
Cairo
Mansoura
Zakazic

Example 7.33 Use of INTERSECT

*Construct a list of all cities where there is **both** a branch office and a property.*

```
(SELECT city
FROM Branch)
INTERSECT
(SELECT city
FROM PropertyForRent);
```

Or

```
(SELECT *
FROM Branch)
INTERSECT CORRESPONDING BY city
(SELECT *
FROM PropertyForRent);
```

This query is executed by producing a result table from the first query and a result table from the second query, and then creating a single result table consisting of those rows that are common to both result tables. The result table is shown in Table 7.33.

Table 7.33 Result table for Example 7.33.

city
Banha

We could rewrite this query without the INTERSECT operator, for example:

```
SELECT DISTINCT b.city
FROM Branch b, PropertyForRent p
WHERE b.city = p.city;
```

Or

```
SELECT DISTINCT city
FROM Branch b
WHERE EXISTS (SELECT *
FROM PropertyForRent p
WHERE b.city =p.city);
```

Ex 7.34 Use of EXCEPT

Construct a list of all cities where there is a branch office but no properties.

```
(SELECT city
FROM Branch)
EXCEPT
(SELECT city
FROM PropertyForRent);
```

Or

```
(SELECT *
FROM Branch)
(SELECT *
EXCEPT CORRESPONDING BY city
FROM PropertyForRent);
```

This query is executed by producing a result table from the first query and a result table from the second query, and then creating a single result table consisting of those rows that appear in the first result table but not in the second one. The result table is shown in Table 7.34.

Table 7.34 Result table for Example 7.34.

city
Cairo

We could rewrite this query without the EXCEPT operator, for example:

```
SELECT DISTINCT city
FROM Branch
WHERE city NOT IN (SELECT city
```

```
FROM PropertyForRent);
```

Or

```
SELECT DISTINCT city
FROM Branch b
WHERE NOT EXISTS
(SELECT *
FROM PropertyForRent p
WHERE b.city □□p.city);
```